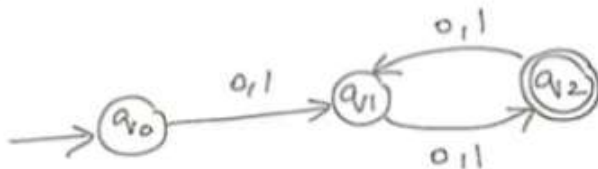


CSA1302 – Theory Of Computation (Design Task)

1.

1.Design the regular expression for the language represented by the given automata



SOLUTION:

Step 1: Analyze the Automaton Structure

Initial State: q_0

Intermediate State: q_1

Final/Accepting State: q_2 (indicated by the double circle)

Step 2: Trace the Paths to the Accepting State

Initial move from q_0 to q_1 :

Transition symbol: 0 or 1, which is represented as $(0 + 1)$.

Move from q_1 to q_2 :

Transition symbol: 0 or 1, which is represented as $(0 + 1)$.

Looping between q_2 and q_1 :

From q_2 , we can go back to q_1 on 0 or 1 $\rightarrow (0 + 1)$.

From q_1 , we must return to q_2 on 0 or 1 $\rightarrow (0 + 1)$.

Therefore, one complete loop cycle from q_2 back to q_2 consumes two symbols:

$(0 + 1)(0 + 1)$

Since this loop can repeat zero or more times, it is represented as:

$((0 + 1)(0 + 1))^*$

Step 3: Combine the Path Expressions

Combining the initial path to q2 and the repeating loops:

Regular Expression = $(0 + 1)(0 + 1)((0 + 1)(0 + 1))^*$

Alternative Form:

This expression describes any string over the alphabet $\{0, 1\}$ that has an even length greater than zero (i.e., 2, 4, 6, 8, ...).

Regular Expression = $((0 + 1)(0 + 1))^+$

2.

2. Consider the following grammar

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb | e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

SOLUTION:

Given Grammar (where e represents epsilon):

$S \rightarrow aSc \mid T \mid U \mid e$

$T \rightarrow bTc \mid e$

$U \rightarrow aUb \mid e$

PART A: What is the language defined by the grammar?

Step 1: Break down each non-terminal

Production $T \rightarrow bTc \mid \epsilon$:

Generates equal numbers of b's followed by c's.

$$L(T) = \{ b^m c^m \mid m \geq 0 \}$$

Production $U \rightarrow aUb \mid \epsilon$:

Generates equal numbers of a's followed by b's.

$$L(U) = \{ a^n b^n \mid n \geq 0 \}$$

Production $S \rightarrow aSc \mid T \mid U \mid \epsilon$:

S can recursively wrap matching a's and c's around T or U.

Applying $S \rightarrow aSc$ k times before choosing T:

$$\begin{aligned} a^k T c^k &= a^k (b^m c^m) c^k \\ &= a^k b^m c^{(m+k)}, \text{ where } k, m \geq 0 \end{aligned}$$

Applying $S \rightarrow aSc$ k times before choosing U:

$$\begin{aligned} a^k U c^k &= a^k (a^n b^n) c^k \\ &= a^{(n+k)} b^n c^k, \text{ where } k, n \geq 0 \end{aligned}$$

Step 2: Formal Definition of the Language

$$L(G) = \{ a^k b^m c^{(m+k)} \mid k, m \geq 0 \}$$

U

$$\{ a^{(n+k)} b^n c^k \mid k, n \geq 0 \}$$

PART B: Give the derivation tree for the word aaabbc

For the string $w = aaabbc$:

Number of a's = 3

Number of b's = 2

Number of c's = 1

Here, notice that:

Number of a's = Number of b's + Number of c's

$$3 = 2 + 1$$

This fits the pattern:

$$a^{(n+k)} b^n c^k$$

where $n = 2$ and $k = 1$.

Derivation Steps (Leftmost Derivation):

$$S \Rightarrow aSc$$

(using $S \rightarrow aSc$)

$$\Rightarrow aUc$$

(using $S \rightarrow U$)

$$\Rightarrow a(aUb)c$$

(using $U \rightarrow aUb$)

$$\Rightarrow aaUbc$$

(using $U \rightarrow aUb$)

$$\Rightarrow aa(aUb)bc$$

(using $U \rightarrow aUb$)

$$\Rightarrow aaaUbbc$$

$$\Rightarrow aaa(e)bbc$$

(using $U \rightarrow e$)

$$\Rightarrow aaabbc$$

Parse Tree Structure :



Reading the leaf nodes from left to right:

$a + a + a + b + b + c$

Therefore:

aaabbc

PART C: Is the above grammar ambiguous? Justify.

Answer:

Yes, the grammar is ambiguous.

Justification:

A grammar is ambiguous if there exists at least one string in the language that has two or more distinct derivation trees (or leftmost derivations).

Consider the empty string ϵ (epsilon):

Derivation Tree 1:

$S \Rightarrow \epsilon$

Derivation Tree 2:

$S \Rightarrow T \Rightarrow \epsilon$

Derivation Tree 3:

$S \Rightarrow U \Rightarrow \epsilon$

Since the string ϵ has multiple distinct derivation trees, the grammar is ambiguous.

Therefore, the given grammar is ambiguous.

3.

3. Design a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression 00^*1+11^*0

Step 1: Breakdown the Regular Expression

The regular expression is:

$$r = r1 + r2$$

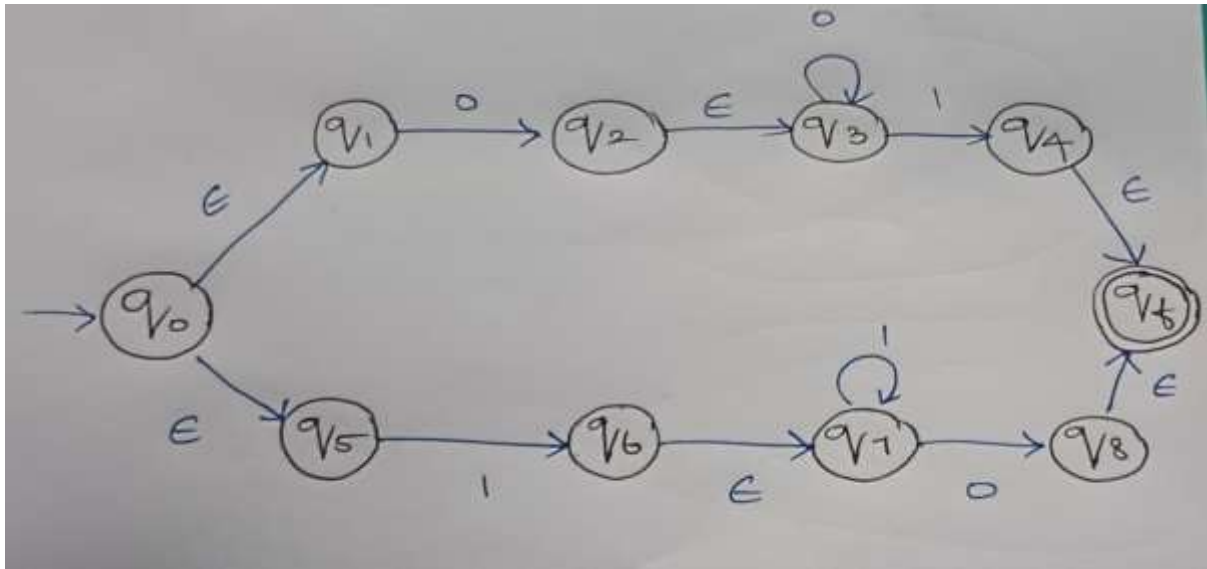
where:

1. $r1 = 00^*1$ (An initial 0, followed by zero or more 0s, ending with a 1)
2. $r2 = 11^*0$ (An initial 1, followed by zero or more 1s, ending with a 0)

Step 2: Transition Table for the Epsilon-NFA

State	Input 0	Input 1	Input epsilon	Description
q0 (Start)	$\emptyset$	$\emptyset$	{q1, q5}	Start state (branches to r1 and r2)
q1	{q2}	$\emptyset$	$\emptyset$	Reads initial 0 for r1
q2	$\emptyset$	$\emptyset$	{q3}	Entrance to 0^* loop
q3	{q3}	{q4}	$\emptyset$	Self-loop on 0 (0^*), moves to q4 on 1
q4	$\emptyset$	$\emptyset$	{qf}	Reaches Final State
q5	$\emptyset$	{q6}	$\emptyset$	Reads initial 1 for r2
q6	$\emptyset$	$\emptyset$	{q7}	Entrance to 1^* loop
q7	{q8}	{q7}	$\emptyset$	Self-loop on 1 (1^*), moves to q8 on 0
q8	$\emptyset$	$\emptyset$	{qf}	Reaches Final State
qf (Final)	$\emptyset$	$\emptyset$	$\emptyset$	Accepting State

Step 3: State Diagram Representation



Top Path (00^*1):

Transition from start state q_0 to q_1 using epsilon.

$q_0 \xrightarrow{\epsilon} q_1$

Transition from q_1 to q_2 using 0, then q_2 to q_3 using epsilon.

$q_1 \xrightarrow{0} q_2 \xrightarrow{\epsilon} q_3$

Loop at q_3 using 0 (handles 0^*).

$q_3 \xrightarrow{0} q_3$

Exit transition from q_3 to q_4 using 1, then q_4 to q_f using epsilon.

$q_3 \xrightarrow{1} q_4 \xrightarrow{\epsilon} q_f$

Bottom Path (11^*0):

Transition from start state q_0 to q_5 using epsilon.

$q_0 \xrightarrow{\epsilon} q_5$

Transition from q_5 to q_6 using 1, then q_6 to q_7 using epsilon.

$q_5 \xrightarrow{1} q_6 \xrightarrow{\epsilon} q_7$

Loop at q_7 using 1 (handles 1^*).

$q_7 \xrightarrow{1} q_7$

Exit transition from q_7 to q_8 using 0, then q_8 to q_f using epsilon.

$q_7 \xrightarrow{0} q_8 \xrightarrow{\epsilon} q_f$

4. i) Show that if L_1 and L_2 are regular then $(L_1 + L_2)$ also regular

ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

4.

SOLUTION:

Statement:

Regular languages are closed under the union operation ($L_1 \cup L_2$, also written as $L_1 + L_2$). That is, if L_1 and L_2 are regular languages, then $L_1 \cup L_2$ is also a regular language.

Proof Method 1: Using Finite Automata (DFA / NFA Construction)

Since L_1 and L_2 are regular, there exist Deterministic Finite Automata (DFAs) that accept them:

$M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ such that $L(M_1) = L_1$

$M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ such that $L(M_2) = L_2$

Assume without loss of generality that Q_1 and Q_2 are disjoint.

We can construct a new Non-Deterministic Finite Automaton with epsilon-transitions:

$M = (Q, \Sigma, \delta, q_0, F)$

that accepts $L_1 \cup L_2$.

States:

$Q = Q_1 \cup Q_2 \cup \{q_0\}$

where q_0 is a new start state.

Start State:

q_0

Accepting States:

$F = F_1 \cup F_2$

Transition Function δ :

$\delta(q_0, \epsilon) = \{q_{01}, q_{02}\}$

This spawns non-deterministic choices to both automata.

$\delta(q, a) = \delta_1(q, a)$ for all q in Q_1 , a in Σ

$\delta(q, a) = \delta_2(q, a)$ for all q in Q_2 , a in Σ

Since we constructed an epsilon-NFA M such that:

$L(M) = L_1 \cup L_2$

the language $(L_1 + L_2)$ is regular.

Proof Method 2: Using Regular Expressions

By definition of regular languages, if L_1 and L_2 are regular, there exist regular expressions R_1 and R_2 such that:

$L(R_1) = L_1$

$L(R_2) = L_2$

By the formal definition of regular expressions, if R_1 and R_2 are valid regular expressions, then:

$(R_1 + R_2)$

is also a valid regular expression.

The language defined by $(R_1 + R_2)$ is:

$L(R_1 + R_2) = L(R_1) \cup L(R_2)$

$= L_1 \cup L_2$

Since $(R_1 + R_2)$ is a valid regular expression, the language $(L_1 + L_2)$ must be regular.

Therefore, regular languages are closed under the union operation.

ii) Is the language $L = \{ a^p \mid p \text{ is prime} \}$ regular? Justify.

Answer:

No, the language $L = \{ a^p \mid p \text{ is prime} \}$ is not regular.

Justification using the Pumping Lemma for Regular Languages:

1. Assume for contradiction that L is regular.

By the Pumping Lemma, there exists a pumping length $p \geq 1$ such that any string w in L with $|w| \geq p$ can be split into three parts:

$w = xyz$

satisfying:

$|y| > 0$ (that is, y is not equal to epsilon)

$|xy| \leq p$

$xy^i z$ is in L for all $i \geq 0$

2. Choose a suitable string w in L .

Let k be a prime number such that $k \geq p$.

Choose:

$w = a^k$

Since k is prime:

w is in L

and:

$|w| = k \geq p$

3. Analyze the decomposition $w = xyz$.

Since $w = a^k$, all characters are a 's.

Let:

$|x| = r$

$|y| = m$, where $m \geq 1$ because $|y| > 0$

$$|z| = s$$

Thus:

$$|w| = r + m + s = k$$

where k is a prime number.

4. Apply the pumping condition ($xy^i z$ is in L).

Let us pump y by choosing:

$$i = k + 1$$

Then:

$$|xy^{(k+1)}z| = |x| + (k+1)|y| + |z|$$

$$= r + (k+1)m + s$$

$$= (r + m + s) + km$$

$$= k + km$$

$$= k(1 + m)$$

5. Arrive at a contradiction.

Since $m \geq 1$:

$$1 + m \geq 2$$

Since k is a prime number and $k \geq 2$, the length: $|xy^{(k+1)}z| = k(1 + m)$ is a composite number. It has factors k and $(1 + m)$.

Therefore:

$xy^{(k+1)}z = a^{k(1+m)}$ does not have a prime number of a 's.

Hence: $xy^{(k+1)}z$ is not in L .

This contradicts condition (3) of the Pumping Lemma, which states that: $xy^i z$ must be in L for all $i \geq 0$.

Conclusion:

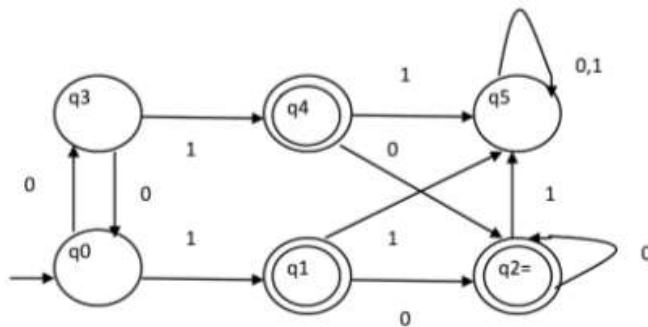
The assumption that L is regular is false.

Hence:

$L = \{ a^p \mid p \text{ is prime} \}$ is not regular.

5.

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



SOLUTION:

Step 1: Extract the Original DFA Information

Set of States:

$$Q = \{q0, q1, q2, q3, q4, q5\}$$

Alphabet:

$$\Sigma = \{0, 1\}$$

Initial State:

q0

Set of Final/Accepting States:

$$F = \{q1, q2, q4\}$$

Set of Non-Final States:

$$Q \setminus F = \{q0, q3, q5\}$$

Original Transition Table:

State	Input 0	Input 1	Is Final?
$\rightarrow q_0$	q_3	q_1	No
$*q_1$	q_2	q_5	Yes
$*q_2$	q_2	q_5	Yes
q_3	q_0	q_4	No
$*q_4$	q_2	q_5	Yes
q_5	q_5	q_5	No

Step 2: State Minimization using Equivalence Partitioning (Table-Filling / Myhill-Nerode Method)

Iteration P0 (Partition into Final and Non-Final States):

$$P_0 = \{ \{q_0, q_3, q_5\}, \{q_1, q_2, q_4\} \}$$

Let:

$$G_1 = \{q_0, q_3, q_5\} \text{ (Non-Final States)}$$

$$G_2 = \{q_1, q_2, q_4\} \text{ (Final States)}$$

Iteration P1:

1. Check group $G_1 = \{q_0, q_3, q_5\}$:

On input 0:

$$\delta(q_0, 0) = q_3, \text{ which belongs to } G_1$$

$$\delta(q_3, 0) = q_0, \text{ which belongs to } G_1$$

$$\delta(q_5, 0) = q_5, \text{ which belongs to } G_1$$

On input 1:

$$\delta(q_0, 1) = q_1, \text{ which belongs to } G_2$$

$$\delta(q_3, 1) = q_4, \text{ which belongs to } G_2$$

$\delta(q_5, 1) = q_5$, which belongs to G_1

Notice that on input 1, states q_0 and q_3 go to group G_2 , while q_5 stays in group G_1 .

Therefore, q_5 is not equivalent to q_0 and q_3 .

Splitting G_1 yields:

$\{q_0, q_3\}$ and $\{q_5\}$

2. Check group $G_2 = \{q_1, q_2, q_4\}$:

On input 0:

$\delta(q_1, 0) = q_2$, which belongs to G_2

$\delta(q_2, 0) = q_2$, which belongs to G_2

$\delta(q_4, 0) = q_2$, which belongs to G_2

On input 1:

$\delta(q_1, 1) = q_5$, which belongs to G_1

$\delta(q_2, 1) = q_5$, which belongs to G_1

$\delta(q_4, 1) = q_5$, which belongs to G_1

All states in G_2 transition to the exact same groups for both inputs 0 and 1.

Thus, q_1 , q_2 , and q_4 are equivalent.

$P_1 = \{ \{q_0, q_3\}, \{q_5\}, \{q_1, q_2, q_4\} \}$

Iteration P_2 :

1. Check group $\{q_0, q_3\}$:

On input 0:

$\delta(q_0, 0) = q_3$

$\delta(q_3, 0) = q_0$

Both stay in the group $\{q_0, q_3\}$.

On input 1:

$$\delta(q_0, 1) = q_1$$

$$\delta(q_3, 1) = q_4$$

Both go to the group $\{q_1, q_2, q_4\}$.

Since both q_0 and q_3 behave identically with respect to the current partitions, they cannot be split further.

$$\Pi_2 = \{ \{q_0, q_3\}, \{q_5\}, \{q_1, q_2, q_4\} \}$$

Since:

$$\Pi_2 = \Pi_1$$

the partitioning algorithm terminates.

Step 3: Identify Equivalent States and Merged States

Equivalent Final States:

$$q_1 \equiv q_2 \equiv q_4$$

Therefore, merge them into a single state:

$$[q_1, 2, 4]$$

Equivalent Non-Final States:

$$q_0 \equiv q_3$$

Therefore, merge them into a single state:

$$[q_0, 3]$$

Standalone State:

$$q_5$$

Therefore, retain it as:

$$[q_5]$$

Step 4: Minimized DFA Transition Table

Let:

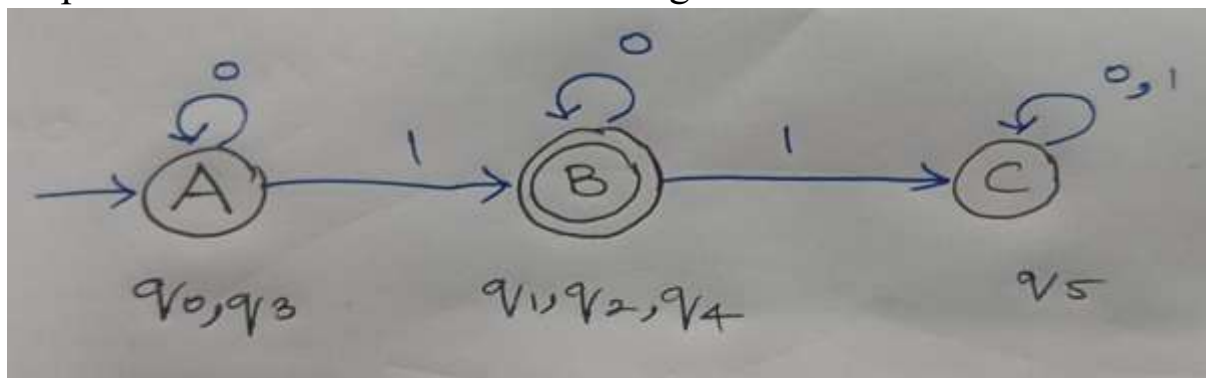
A = [q0,3] (Start State)

B = [q1,2,4] (Final / Accepting State)

C = [q5] (Non-Final / Dead State)

Minimized State	Composition	Input 0	Input 1
$\rightarrow A$	$\{q_0, q_3\}$	A	B
$*B$	$\{q_1, q_2, q_4\}$	B	C
C	$\{q_5\}$	C	C

Step 5: Minimized Automata State Diagram



Description of Transitions for Drawing:

State A (q0, q3):

- Initial state.
- Self-loop on input 0: $\delta(A, 0) = A$
- Transition to B on input 1: $\delta(A, 1) = B$

State B (q1, q2, q4):

- Accepting state (double circle).
- Self-loop on input 0: $\delta(B, 0) = B$
- Transition to C on input 1: $\delta(B, 1) = C$

State C (q5):

- Non-Final / Dead State.
- Self-loop on input 0: $\delta(C, 0) = C$
- Self-loop on input 1: $\delta(C, 1) = C$